📙 Level 2 Notes — Java Collections, Comparators & Streams

Save as:

`Java_Level_2_Collections_Comparators_Streams.md`

Overview

Level 2 focuses on the Java Collections Framework and introduces the concepts you'll encounter frequently in backend applications and coding interviews.

By the end of this level, you should be comfortable with:

- `List`
- `ArrayList`
- `Set`
- `HashSet`
- `LinkedHashSet`
- `TreeSet`
- `Map`
- `HashMap`
- `LinkedHashMap`
- `Queue`
- `ArrayDeque`
- `PriorityQueue`
- Min Heap
- Max Heap
- `Comparator`
- Multi-level sorting
- `LinkedHashMap` LRU cache
- Java Streams
- `groupingBy`
- `summingDouble`
- `counting`
- `Optional`

1. ArrayList

```
List<String> languages =
        new ArrayList<>();
```

Common operations:

```
languages.add("Java");

languages.remove("Python");

languages.set(1, "Rust");

languages.contains("Java");

languages.get(0);

languages.size();
```

Mental model

```
ArrayList
    ↓
Ordered collection
    ↓
Allows duplicates
    ↓
Index based
```

---

2. HashSet

A `HashSet` stores unique values.

```
Set<Integer> numbers =
        new HashSet<>();
```

Example:

```
Set<Integer> numbers =
        new HashSet<>(
            Arrays.asList(
                10, 20, 10, 30, 20
```

```
                10, 20, 10, 30, 20
            )
        );
```

Result:

```
[10, 20, 30]
```

Mental model

```
HashSet
→ uniqueness
→ no guaranteed ordering
```

---

3. LinkedHashSet

```
Set<String> names =
        new LinkedHashSet<>(namesList);
```

Provides:

```
uniqueness
+
insertion order
```

---

4. TreeSet

```
Set<Integer> numbers =
        new TreeSet<>(numbersList);
```

Provides:

```
uniqueness
+
sorted order
```

Collection comparison

Collection comparison

Collection	Duplicate	Ordering
`HashSet`	❌	None guaranteed
`LinkedHashSet`	❌	Insertion
`TreeSet`	❌	Sorted

---

5. Finding Duplicates

```
Set<Integer> seen = new HashSet<>();
Set<Integer> duplicates = new HashSet<>();

for (int num : numbers) {

    if (seen.contains(num)) {
        duplicates.add(num);
    } else {
        seen.add(num);
    }
}
```

A useful optimization:

```
if (!seen.add(num)) {
    duplicates.add(num);
}
```

Because `Set.add()` returns `false` when the element already exists.

---

6. LinkedHashMap

A `LinkedHashMap` maintains insertion order.

```
Map<String, Integer> map =
        new LinkedHashMap<>();
```

This becomes particularly useful for:

- Ordered maps

- Frequency problems
- LRU caches

---

7. Queue

```
Queue<String> queue =
        new ArrayDeque<>();
```

Common operations:

```
queue.offer("Java");
queue.poll();
queue.peek();
```

Mental model

```
FIFO

First In
   ↓
First Out
```

---

8. PriorityQueue

`PriorityQueue` is Java's heap implementation.

Default:

```
PriorityQueue<Integer> minHeap =
        new PriorityQueue<>();
```

This gives a **Min Heap**.

For Max Heap:

```
PriorityQueue<Integer> maxHeap =
        new PriorityQueue<>(
            Collections.reverseOrder()
        );
```

9. Top K Using Min Heap

One of the most useful DSA patterns.

```
PriorityQueue<Integer> minHeap =
        new PriorityQueue<>();

for (int num : numbers) {

    minHeap.offer(num);

    if (minHeap.size() > k) {
        minHeap.poll();
    }
}
```

Why it works

If `k = 3`, the heap never keeps more than three elements.

The smallest element is removed whenever size exceeds `3`.

Therefore the heap contains the:

```
3 largest elements
```

Complexity:

```
O(n log k)
```

10. Top K Using Max Heap

```
PriorityQueue<Integer> maxHeap =
        new PriorityQueue<>(
            Collections.reverseOrder()
        );

for (int num : numbers) {
    maxHeap.offer(num);
}
```

Then:

```
for (int i = 0; i < k; i++) {

    if (!maxHeap.isEmpty()) {
        result.add(maxHeap.poll());
    }
}
```

Complexity:

```
O(n + k log n)
```

---

11. Comparator

Comparator controls sorting.

Example:

```
employees.sort(
    Comparator.comparingDouble(
        employee -> employee.salary
    )
);
```

Ascending salary.

Descending:

```
employees.sort(
    Comparator
        .comparingDouble(
            employee -> employee.salary
        )
        .reversed()
);
```

---

12. Multiple Sorting Conditions

Example:

```
employees.sort(
    Comparator
        .comparingDouble(
            (Employee employee) -> employee.salary
        )
        .reversed()
        .thenComparing(
            employee -> employee.name
        )
);
```

Meaning:

```
salary → descending
name   → ascending
```

---

13. Frequency Ranking with PriorityQueue

First count:

```
Map<String, Integer> frequencyMap =
        new HashMap<>();

for (String word : languages) {

    frequencyMap.put(
        word,
        frequencyMap.getOrDefault(word, 0) + 1
    );
}
```

Then create a frequency-based heap:

```
PriorityQueue<String> maxHeap =
        new PriorityQueue<>(
            (a, b) -> Integer.compare(
                frequencyMap.get(b),
                frequencyMap.get(a)
            )
        );
```

Important:

Prefer:

```
Integer.compare(a, b)
```

over:

```
a - b
```

because subtraction can overflow.

---

14. LRU Cache with LinkedHashMap

A very useful Java pattern:

```
private final Map<String, String> cache =
        new LinkedHashMap<>(
            3,
            0.75f,
            true
        ) {

    @Override
    protected boolean removeEldestEntry(
            Map.Entry<String, String> eldest) {

        return size() > 3;
    }
};
```

The important argument:

```
true
```

means:

```
accessOrder = true
```

So recently accessed entries move toward the end.

This creates an LRU-style cache.

---

15. Traditional Map Aggregation

Example:

```
Map<String, Double> amountMap =
        new HashMap<>();

for (Transaction txn : transactions) {

    amountMap.put(
        txn.userId,
        amountMap.getOrDefault(
            txn.userId,
            0.0
        ) + txn.amount
    );
}
```

This pattern is extremely useful.

Remember:

```
getOrDefault()
+
put()
```

---

16. Java Streams

Streams allow collections to be processed declaratively.

Example:

```
double total =
        transactions.stream()
            .filter(
                txn -> "SUCCESS"
                    .equals(txn.status)
            )
            .mapToDouble(
                txn -> txn.amount
            )
            .sum();
```

Flow:

```
Collection
   ↓
stream()
   ↓
filter()
   ↓
mapToDouble()
   ↓
sum()
```

---

17. `filter()`

Filters elements.

```
transactions.stream()
    .filter(
        txn -> "SUCCESS"
            .equals(txn.status)
    );
```

Only successful transactions continue through the pipeline.

---

18. `map()`

Transforms elements.

Example:

```
List<String> names =
        employees.stream()
            .map(employee -> employee.name)
            .toList();
```

Concept:

```
Employee
   ↓
name
```

19. `mapToDouble()`

Useful for numerical aggregation.

```
transactions.stream()
    .mapToDouble(txn -> txn.amount)
    .sum();
```

20. `max()` + Optional

Find highest transaction:

```
return transactions.stream()
        .max(
            Comparator.comparingDouble(
                txn -> txn.amount
            )
        )
        .map(
            txn ->
                txn.userId +
                " → " +
                txn.amount
        )
        .orElse("No transactions");
```

Important concept:

```
max()
 ↓
Optional
 ↓
map()
 ↓
orElse()
```

21. `groupingBy()`

Group data by a field

Group data by a field.

```
Map<String, List<Employee>> result =
        employees.stream()
            .collect(
                Collectors.groupingBy(
                    employee -> employee.department
                )
            );
```

Result conceptually:

```
Engineering → [Ramu, Alex]
Sales       → [John, Mike]
HR          → [Sam]
```

---

22. `groupingBy()` + `summingDouble()`

Calculate total amount per user:

```
Map<String, Double> result =
        transactions.stream()
            .collect(
                Collectors.groupingBy(
                    txn -> txn.userId,
                    Collectors.summingDouble(
                        txn -> txn.amount
                    )
                )
            );
```

Concept:

```
group by user
      ↓
sum amounts
```

---

23. `groupingBy()` + `counting()`

Count successful transactions per user:

```
Map<String, Long> result =
        transactions.stream()
            .filter(
                txn -> "SUCCESS"
                    .equals(txn.status)
            )
            .collect(
                Collectors.groupingBy(
                    txn -> txn.userId,
                    Collectors.counting()
                )
            );
```

---

24. `computeIfAbsent()`

Useful for grouping manually.

Instead of:

```
if (employeeGroup.containsKey(
        employee.department)) {

    employeeGroup
        .get(employee.department)
        .add(employee);

} else {

    employeeGroup.put(
        employee.department,
        new ArrayList<>()
    );
}
```

Use:

```
employeeGroup
    .computeIfAbsent(
        employee.department,
        k -> new ArrayList<>()
    )
    .add(employee);
```

Mental model:

```
If key exists
    → return existing value

If key doesn't exist
    → create value
    → put it in map
    → return it
```

---

25. Collection Cheat Sheet

```
List
 └── ArrayList
       → ordered
       → duplicates allowed
       → index based

Set
 ├── HashSet
 │     → unique
 │
 ├── LinkedHashSet
 │     → unique + insertion order
 │
 └── TreeSet
       → unique + sorted

Map
 ├── HashMap
 │     → key/value
 │
 └── LinkedHashMap
       → key/value + insertion order

Queue
 └── ArrayDeque
       → FIFO

PriorityQueue
 ├── Min Heap
 └── Max Heap
```

---

26. Level 2 — Most Important Patterns

Frequency counting

```
map.put(
    key,
    map.getOrDefault(key, 0) + 1
);
```

Duplicate detection

```
if (!set.add(value)) {
    duplicates.add(value);
}
```

Grouping

```
map.computeIfAbsent(
    key,
    k -> new ArrayList<>()
).add(value);
```

Sorting

```
Comparator.comparing(...)
```

Reverse sorting

```
.reversed()
```

Multiple sorting fields

```
.thenComparing(...)
```

Top K

```
PriorityQueue
```

Stream filtering

```
.filter(...)
```

Stream transformation

```
.map(...)
```

Grouping streams

```
Collectors.groupingBy(...)
```

Aggregation

```
Collectors.summingDouble(...)
Collectors.counting()
```

🎯 Level 2 Final Checklist

```
✅ ArrayList
✅ HashSet
✅ LinkedHashSet
✅ TreeSet
✅ HashMap
✅ LinkedHashMap
✅ Queue
✅ ArrayDeque
✅ PriorityQueue
✅ Min Heap
✅ Max Heap
✅ Comparator
✅ reversed()
✅ thenComparing()
✅ Frequency counting
✅ Duplicate detection
✅ computeIfAbsent()
✅ LRU Cache
✅ Streams
✅ filter()
✅ map()
✅ mapToDouble()
✅ max()
✅ Optional
✅ groupingBy()
✅ summingDouble()
✅ counting()
```

Recommended files

Your Java notes folder can now look like:

```
Java/
│
├── Java_Learning_Roadmap.md
│
├── Java_Level_1_Basics_Coding_Challenges.md
│
├── Java_Level_2_Collections_Comparators_Streams.md
│
└── Java_Level_3_OOP_Core_Java.md
```